

Critical Section Problem || Race condition in OS

1. Why we need Process Synchronization?

When many threads/processes work together and share the same data, they may try to update it at the same time.

If this is not controlled, the final result becomes wrong.

So, **synchronization keeps shared data correct and consistent.**

2. What is a Critical Section?

It is the part of a program where shared resources are accessed — like:

- shared variable
- shared file
- shared database

Only **one process/thread should be inside this critical section at a time**, otherwise results may become incorrect.

3. Major Thread Scheduling Issue — Race Condition

A **race condition** happens when:

- Two or more threads access the same shared data
- Both try to modify it at the same time
- Final result depends on **which thread executes first**

They literally *race* with each other to update data.

Real-World Example of Race Condition

Imagine two people withdrawing money from the same bank account:

Account balance = ₹1000

Person A	Person B
reads balance = 1000	reads balance = 1000
withdraws 700 → writes 300	withdraws 700 → writes 300

Both read 1000 at the same time, both calculate new balance correctly from their point of view.

Final result = ₹300 (not ₹-400 or correct sequential deduction) → **Incorrect result**

Because both worked together without coordination → **Race Condition**

4. How to Solve Race Condition

Method	Meaning
Atomic operation	Critical part runs completely in one go, no interruption
Mutex/Locks	Allow only one thread at a time inside critical section
Semaphore	A counter-based synchronization tool, controls access to resources

5. Can we solve race condition using simple flag variable?

No, because one thread may still ignore the flag or both may set the flag at the same time. It does not ensure strict mutual exclusion.

6. Peterson's Solution

A classic theoretical method that prevents race condition, but it **works only for 2 processes/threads**, not more.

7. Mutex / Locks

A lock ensures only one thread enters the critical section. Others must wait until the lock is released.

Benefits:

Prevents race condition
Safe and simple to use

Disadvantages:

1. **Contention** — Only one thread can run, others keep waiting.
2. **Deadlock** — If a thread acquires a lock and never releases it, all others get stuck forever.
3. **Debugging is difficult** — Hard to find which thread blocked others.

Real-world example for Locks

Think of a washroom with one key:

Only one person can use it (critical section)

Whoever holds the key first can enter (lock acquired)

Others wait outside (contention)

If the person goes out with the key or doesn't come back → others wait forever (deadlock)

This is exactly how Mutex/Locks behave in operating systems.